

VibeCodeBench Manual QC Process

How to review a task by hand before it moves to approved, whatever the automation says.

TL;DR. AutoQC and the oracle are not always correct, so every task gets a rigorous manual pass on top of them. When you pick up a PR the automation has gated green, work the task from its files and from the running app, and confirm the AutoQC result is actually true.

Scope

Review every task before it moves to approved, and review the whole task:

- `instruction.md`
- every criterion in `tests/workflows/workflows.toml`
- the reference app in `solution/app/`

Before you start, confirm the checks are current

A PR opened through the API may not have triggered AutoQC, static checks or trials, and with the recent volume of changes that is common. If task checks are missing, fill them in with `/ci`, `/autoqc` and `/oracle`.

- A green sticky may belong to an older commit. Check the commit it names, and rerun against the current head.
- Use the parameterized check, such as `stagehand (tasks/approved/x)`, rather than the bare matrix name when a run was canceled.
- A reward of exactly `0.0` in under two minutes usually means the app never booted. Rerun the checks.

Once the checks are current, start the review.

The five passes

Pass	What you read	What it catches
Pass 1. Read the instruction	<code>instruction.md</code>	A brief that cannot be built from
Pass 2. Compare with the verifier	<code>instruction.md</code> and <code>workflows.toml</code>	Graded behavior the brief never states
Pass 3. Check the criteria hold up	<code>workflows.toml</code>	Cross-criterion state, rigid paths, nop-passable substeps

Pass 4. Drive the app	the running app	Dead boots, rules that are not enforced, values that never move
Pass 5. Check declarations	task.toml and the file tree	Dishonest domain, stack or difficulty

Pass 1. Read the instruction

Read `instruction.md` and confirm it is clear and unambiguous, with a complete workspace and build contract: the workspace path, the frontend, the backend, the available services, and how the app is built and run.

Pass 2. Compare the instruction with the verifier

Read `instruction.md` and `tests/workflows/workflows.toml` side by side, and take every criterion in turn.

- **Every graded behavior is stated in the instruction.** If the verifier grades it, the brief has to say it.
- **Every backend identifier is stated or derivable.** It appears in the instruction, or it is the only reasonable choice a competent builder would make.
- **Every asserted value traces to seeded data.** It comes from the fixtures or is derived from them. Prefer a test that makes the app compute a result over one a hardcoded constant would satisfy.

Pass 3. Check that the criteria hold up

Every criterion is graded in its own container, started from a snapshot of the booted app: `docs/per-criterion-verifier.md`, switched on by `per_criterion.enabled` in `run-trials.yml`. A criterion that reads, edits or deletes a record another criterion created, or that assumes a login another criterion performed, fails for every submission. Each criterion has to set up the state it grades and stay decidable on its own.

Look for these recurring problems.

Problem	What it looks like	What to do
Cross-criterion state	A criterion only passes because an earlier one created or changed a record	Have each criterion set up its own state, or use seeded state

Lazy datastore initialization	A database check runs before the app has created the table	Drive the app far enough for the state to exist before checking it
Rigid paths	A criterion requires a URL, route or filename the instruction never fixes	Drop the requirement, or state it in the instruction
Ambiguous wording	A reasonable reader can read a substep two ways	Rewrite it to one interpretation, or split it
Static-value assertions	A substep passes because the app shows a hardcoded value	Change an input and observe the output move
Nop-passable substeps	The golden image or scaffold alone satisfies the substep	Every substep must observe behavior the built app produced

Pass 4. Drive the app

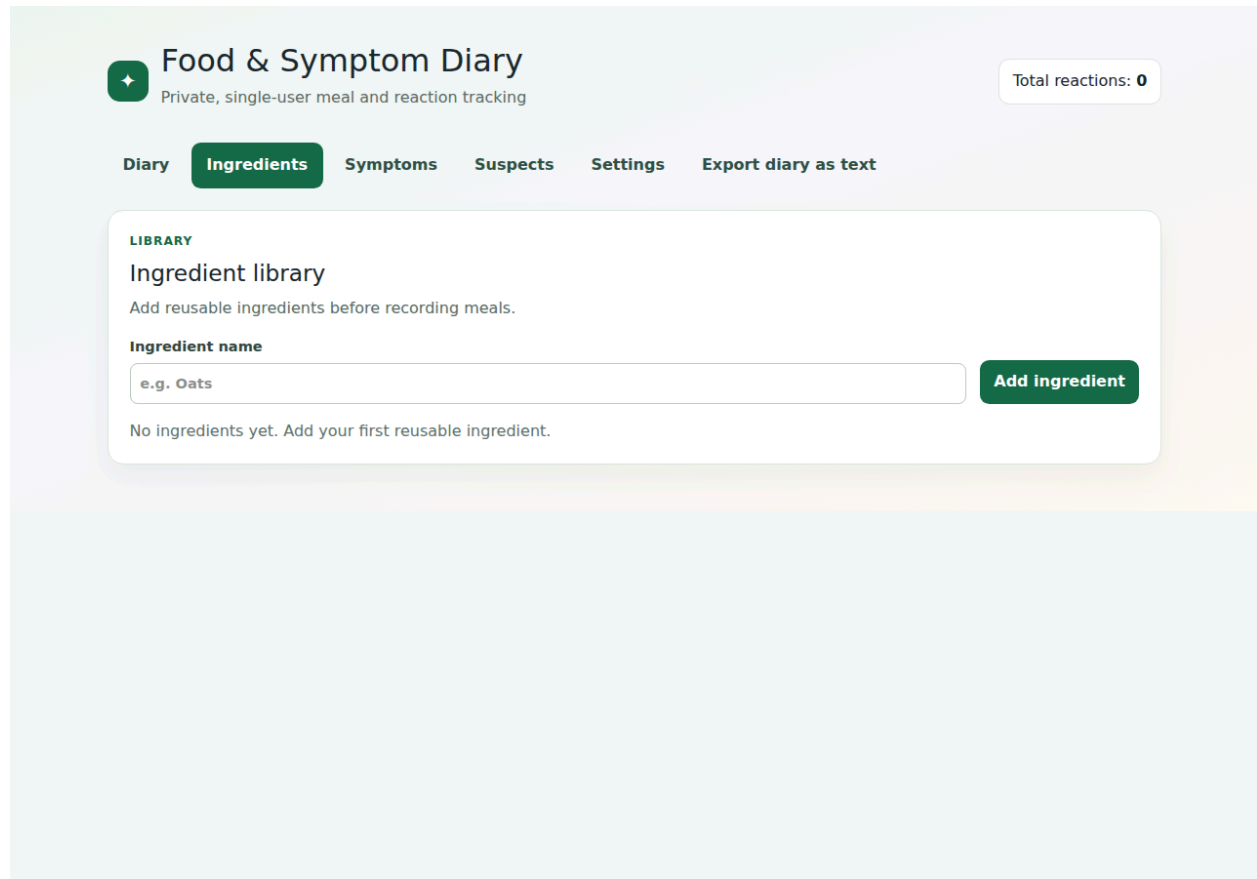
Open the built app and exercise two or three criteria by hand, especially anything Pass 2 or Pass 3 flagged.

- Log in as each seeded role.
- Check the auth boundaries the instruction promises, including redirects, 401s and 403s.
- Pick one computed value and change its input to confirm the output moves.

A few minutes in the running app catches what static review misses.

Worked example: allergy-food-diary

Four frames from one review. The first three are a healthy app; the last is a dead one.



Good task, first load. The app provisions its own session and opens straight onto real content; header reads *Total reactions: 0*. This is what a healthy boot looks like

+

Food & Symptom Diary

Private, single-user meal and reaction tracking

Total reactions: 1

Diary

Ingredients

Symptoms

Suspects

Settings

Export diary as text

DIARY

Meal diary

Record what you ate and the ingredients that were used.

Meal name

e.g. Breakfast

Meal date and time (YYYY-MM-DDTHH:MM)

2026-09-19T00:06

Meal ingredients

☐ Peanuts

Save meal

Saved meals

Each card shows its linked reaction count.

Filter meals by ingredient

All ingredients

Trail mix snack

Sep 19, 2026, 12:00 PM

Ingredients: Peanuts

1 reaction

Edit meal

Good task, computed value moves. After adding an ingredient, a meal, and a linked symptom, the header goes $0 \rightarrow 1$ and the meal card shows 1 reaction.

+

Food & Symptom Diary

Private, single-user meal and reaction tracking

Total reactions: 1

Diary

Ingredients

Symptoms

Suspects

Settings

Export diary as text

DIARY

Meal diary

Record what you ate and the ingredients that were used.

Meal name

Seafood dinner (edited)

Meal date and time (YYYY-MM-DDTHH:MM)

2026-09-19T18:00

Meal ingredients

☒ Shellfish

Save meal changes

Cancel edit

Saved meals

Each card shows its linked reaction count.

Filter meals by ingredient

All ingredients

Seafood dinner

Sep 19, 2026, 6:00 PM

Ingredients: Shellfish

1 reaction

Edit meal

This meal cannot be edited because it has a recorded reaction.

Good task, rule enforced. Editing a meal that has a recorded reaction is refused with a visible reason ("This meal cannot be edited because it has a recorded reaction"), and the saved meal is unchanged. Proof the instruction's rules are actually implemented.

Bad task, dead app. Returns HTTP 200 but paints a blank white screen (framework never mounted). The judge answers "no" to everything and it scores 0.0 in under two minutes

AutoQC can flag false positives

AutoQC is a model grading from the files, and it raises suspicions it cannot always substantiate. A finding is only real if it can quote the artifact that proves it, so an unsupported one is a pass, not a defect. Three from this task:

- `over_specification` **on the Tech Stack section**. AutoQC reads the library names there as over-specified, but that section is the stack declaration, and naming the framework and the state and styling libraries is required. Known noise, ignore it.
- **"Instruction never states X" when X is stated or plainly derivable**. It flagged a graded severity range the instruction does spell out, the 1-to-5 scale. Open the instruction, find the line, override.
- **"Criterion depends on prior state" when the criterion seeds its own state**. The setup is there in an earlier substep. Read the whole criterion before agreeing.

How to record it. Note in your review that you checked the flag and it does not hold, and quote the line that disproves it, so the next reviewer does not re-litigate it. The rule cuts both ways: AutoQC also misses real defects, and every failure above can sit behind a green check, which is the reason this manual pass exists.

Pass 5. Check declarations

Quickly confirm that:

- `[metadata].domain` matches the product the brief describes.
- `[metadata].stack` and `[metadata].backend` are the ones the oracle actually implements.
- `[metadata].rubric_categories` matches the criteria in file order.
- Difficulty is measured, not declared. The band comes from the worst rollout that graded the rubric with zero BLOCKED criteria, and anything above the easy window (`> 0.80`) is `too-easy` and should be rewritten rather than shipped as easy.
- The tree is clean: no `node_modules/`, no `dist/`, no stray fixtures, no second README (`policy/check-shape.sh`).

One-screen checklist

- ☐ Checks ran on the current head, not stale, canceled, or missing because of an API PR
- ☐ Instruction is buildable and has a complete workspace and build contract
- ☐ Every graded behavior is stated in the instruction
- ☐ Every probe identifier (table, collection, path, field, seed) is stated or derivable

- ☐ Each criterion sets up the state it grades and does not lean on another criterion's mutations
- ☐ No lazy-init database issues, rigid paths, ambiguous substeps, or cross-criterion state
- ☐ No substep passes on a nop or a static constant
- ☐ Workflow grammar is valid
- ☐ Oracle is at least 0.9 on the current head, and its failed criteria were reviewed
- ☐ Nop is 0 and the boot contract is green
- ☐ Running app was spot-checked, including one computed value
- ☐ Domain, stack, backend and difficulty declarations are honest
- ☐ Verdict was posted with every defect quoted, located, and paired with a fix